

Professores:

Celso Giusti

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

Fetch API + Integração ao Back-End



O Limite do localStorage

Na Aula 8 o TechFood ganhou memória. Mas experimente abrir o cardápio em outro dispositivo — os pedidos não estão lá.

Por quê? O localStorage é local — vive só no navegador daquele computador.

Para um restaurante real, os pedidos precisam chegar à **cozinha**, independente de qual dispositivo o cliente usou.

Analogia: O localStorage é um bloco de notas no seu bolso. O servidor é o sistema do restaurante — qualquer garçom, em qualquer terminal, vê o mesmo pedido.

O que é a Fetch API?

A **Fetch API** é uma interface nativa do navegador para fazer requisições HTTP — ou seja, para o JavaScript conversar com um servidor.

javascript

```
fetch("http://localhost:3000/produtos")
```

Uma linha. O navegador sai, busca os dados no servidor e traz de volta.

Método HTTP	O que faz	Exemplo
GET	Busca dados	Listar produtos
POST	Cria novo dado	Enviar pedido
PATCH	Atualiza parcialmente	Mudar status
DELETE	Remove	Cancelar pedido

O Problema: o fetch é assíncrono

O JavaScript não para e espera a resposta do servidor — ele continua rodando. Isso se chama **operação assíncrona**.

// Sem esperar: o log roda antes dos dados chegarem!

```
const dados = fetch("http://localhost:3000/produtos");
```

```
console.log(dados); // Promise { pending } — ainda não chegou!
```

Solução — .then() (forma antiga):

```
fetch("http://localhost:3000/produtos")
```

```
.then(res => res.json())
```

```
.then(dados => console.log(dados));
```

async/await: a forma moderna

async/await é a forma moderna de esperar uma Promise sem travar a página:

```
async function buscar() {  
  const res = await fetch("http://localhost:3000/produtos");  
  const dados = await res.json();  
  console.log(dados); // dados reais!  
}buscar();
```

- **async** → transforma a função em assíncrona
- **await** → pausa a função e espera a resposta chegar
- A página **não trava** — só a função espera

Lembre-se: **async** transforma a função em assíncrona. **await** pausa a função e espera a Promise resolver — sem travar a página.

Tratamento de Erros com try/catch

Qualquer coisa pode dar errado: servidor offline, sem internet, rota inexistente. Sem tratamento, o erro quebra a página silenciosamente.

```
async function buscarProdutos() {  
  try {  
    const res = await fetch("http://localhost:3000/produtos");  
    if (!res.ok) throw new Error("Erro no servidor: " + res.status);  
    const dados = await res.json();  
    return dados;  
  } catch (erro) {  
    console.error("Falha:", erro.message);  
  }  
}
```

Atenção: fetch não lança erro em respostas 404 ou 500 — só em falha de rede. Por isso verificamos **res.ok** manualmente.

Fetch API

CORS: o Inimigo Invisível



Ao fazer o primeiro fetch, você vai ver este erro:

```
Access to fetch at 'http://localhost:3000' from origin  
'http://127.0.0.1:5500' has been blocked by CORS policy
```

CORS bloqueia requisições entre origens diferentes por segurança. Front em **localhost:5500**, back em **localhost:3000** — origens diferentes.

Solução — no servidor (já configurado!):

```
const cors = require('cors');  
app.use(cors());
```

Lembre-se: CORS é resolvido no **servidor**, não no front. O navegador bloqueia, o servidor libera.

sessionStorage: memória da visita

Para guardar o nome do cliente durante a visita usamos **sessionStorage**:

	localStorage	sessionStorage
Dura até	Usuário limpar	Fechar a aba
Uso no TechFood	—	Nome do cliente na mesa

```
sessionStorage.setItem("techfood_cliente", "João");  
const cliente = sessionStorage.getItem("techfood_cliente");
```

Por que sessionStorage aqui? O próximo cliente na mesma mesa não deve ver o nome do anterior. Cada sessão é um cliente novo.

api.js: um lugar só para o fetch

Em vez de espalhar **fetch** por todos os arquivos, centralizamos em **api.js**:

```
const API_URL = "http://localhost:3000";

async function requisitar(endpoint, opcoes) {
  const response = await fetch(API_URL + endpoint, opcoes);
  const dados = await response.json();
  if (!response.ok) throw new Error(dados.erro);
  return dados;
}

async function buscarProdutos() {
  const resposta = await requisitar("/produtos");
  return resposta.dados;
}
```

Lembre-se: Se a URL do servidor mudar, alteramos só o **API_URL** — em um lugar. O resto do projeto não muda.

Fetch API

api.js: POST



```
async function criarPedido(cliente, itens) {  
  return await requisitar("/pedidos", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ cliente, itens })  
  });  
}
```

- **Content-Type** → diz ao servidor que o body está em JSON
- **JSON.stringify** → converte o objeto para string antes de enviar
- O back-end exige **produto_id** e **quantidade** — não o nome nem o preço

api.js: PATCH

```
async function atualizarStatusPedido(id, status) {  
  return await requisitar("/pedidos/" + id + "/status", {  
    method: "PATCH",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ status })  
  });  
}
```

- **PATCH** → atualização **parcial** — só o status, não o pedido inteiro
- **PUT** atualizaria o recurso inteiro — aqui não é o caso
- Fluxo: **pendente** → **preparo** → **pronto** → **entregue**

Resumo

- ✓ **Fetch API** = Interface nativa para requisições HTTP ao servidor.
- ✓ **GET / POST / PATCH** = Métodos HTTP para buscar, criar e atualizar dados.
- ✓ **async/await** = Espera a resposta sem travar a página. Só funciona dentro de função **async**.
- ✓ **try/catch** = Captura erros de rede e respostas inesperadas.
- ✓ **res.ok** = Verifica se o status foi 200–299. **fetch** não lança erro em 404/500.
- ✓ **CORS** = Bloqueio do navegador entre origens diferentes. Resolvido no servidor.
- ✓ **sessionStorage** = Memória da visita — some ao fechar a aba.
- ✓ **api.js** = Arquivo central de comunicação. Um **fetch** por função, um lugar só.
- ✓ **produto_id** = O back-end exige o ID do produto — os cards precisam do **data-id**.

Portal EcoCycle

1

Primeiro fetch no console

Contexto: Portal EcoCycle (Notícias de Ecologia)

Abra o console (F12) com o servidor rodando:

1. `fetch("http://localhost:3000")` — o que retorna?
2. `fetch("http://localhost:3000/produtos").then(r => r.json()).then(d => console.log(d))`
3. Repita usando **async/await** numa função e chame ela
4. Pare o servidor. Tente buscar novamente. O que acontece sem **try/catch**?
5. Adicione **try/catch** e mostre mensagem amigável quando falhar

Portal EcoCycle

2

EcoCycle com fetch – Buscar notícias:

Contexto: Portal EcoCycle (Notícias de Ecologia)

1. Crie **async buscarNoticias()** com **try/catch**
2. Faça fetch para **http://localhost:3000/noticias**
3. Use os dados para criar cards com **createElement**
4. Se der erro, exiba: *"Não foi possível carregar as notícias"*

Portal EcoCycle

3

EcoCycle com fetch – Enviar sugestão:

Contexto: Portal EcoCycle (Notícias de Ecologia)

1. No **submit** do formulário, adicione um **fetch** com **POST**
2. Body: **JSON.stringify({ titulo, descricao })**
3. Header: **"Content-Type": "application/json"**
4. Se falhar, ainda crie o card na tela

Portal EcoCycle

4

Erros e sessionStorage – Simulando erros:

Contexto: Portal EcoCycle (Notícias de Ecologia)

1. Pare o servidor → o **catch** captura?
2. Acesse uma rota inexistente → **res.ok** é **false**?
3. Remova o **cors()** → o que aparece no console?
4. Para cada cenário, exiba uma mensagem diferente na tela
5. **Desafio:** diferença entre erro no **catch** e erro verificado pelo **res.ok**?

Portal EcoCycle

5

Erros e sessionStorage – sessionStorage:

Contexto: Portal EcoCycle (Notícias de Ecologia)

1. Ao carregar, verifique **sessionStorage.getItem("eco_usuario")**
2. Se não existir, exiba modal pedindo o nome
3. Salve e exiba no header: *"Bem-vindo, João! 🌿"*
4. Navegue entre páginas — o nome continua? Feche a aba — sumiu?

Referências

MOZILLA. **Fetch API**. MDN Web Docs. Disponível em:

https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API. Acesso em: mai. 2026.

MOZILLA. **async function**. MDN Web Docs. Disponível em:

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/async_function. Acesso em: mai. 2026.

MOZILLA. **try...catch**. MDN Web Docs. Disponível em:

<https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/try...catch>. Acesso em: mai. 2026.

MOZILLA. **Window.sessionStorage**. MDN Web Docs. Disponível em:

<https://developer.mozilla.org/pt-BR/docs/Web/API/Window/sessionStorage>. Acesso em: mai. 2026.

MOZILLA. **Cross-Origin Resource Sharing (CORS)**. MDN Web Docs. Disponível em:

<https://developer.mozilla.org/pt-BR/docs/Web/HTTP/CORS>. Acesso em: mai. 2026.

ANDRADE, Sidnei da Silva. **Criação e interfaces para web HTML5 e CSS3**. São Paulo: SENAI SP Editora, 2017.



ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senaisp.itu

Site

<https://sp.senai.br/unidade/itu/>